1 PROGRAMMING LOOPs: FOR-NEXT & DO-WHILE

It is very often needed, that we want to repeat some part of code several times. In that case we can use programming loops where we can specify which code should be repeated. In general there are two very often situation where we are using the programming loops:

1. We know `how many times` some code should repeat and
2. The code is `repeated` **`while`** `the condition` is met.

1.1 For-Next Loop

So called `For-Next` loop is used whenever the repetition of the code can be controlled by a `counter`. Counter is a number with some **starting value** and gets incremented by each repetition of the code. When `counter` reaches the given **ending value** repetition will stop. Typical examples where `For-Next` loop is used are:

- filling an array of data,
- summarising of all the costs in the bill
- robot should turn for **8 times** with 45 degree step to complete full rotation.

1.2 Do-While Loop

`Do-While` loop is used in situations where we can not predict the numbers of repetitions in advanced. In this case we must state the `condition` that must be met to repeat the code. The repetition of the code will be terminated when the `condition` will not hold anymore. Typical examples are:

- read the content to end of file,
- divide some number by 2 while we can,
- while no obstacle is in front of the robot it should drive forward

1.3 Task: FOR-NEXT LOOP

1. For example the next prog. 1 repeats the functions **robotLeft()** and **robotRight()** for **10 times** and robot will do a funny "dancing" move.

Program 1: Programming Loops.

```
#include "RobotMovingFunctions.h"

void setup()
{
  setIOpins();
  // Repeating Left and Right movement
  // for 10 times to make a danging move
  for (int i = 0; i < 10; i++)
  {
    robotLeft();
    delay(100);
    robotRight();
    delay(100);
  }
  stopTheRobot();
}

void loop()
{

}
```

2. Experiment a bit more with such programming techniques and change some code:
   - value of `i`,
   - duration of `delay()` function,
   - add some other functions to the **`for`**`-next` loop...

1.4 Task: DO-WHILE LOOP

3. Change the **`for`**`-next` loop with this **`do`**-**`while`** loop. Can you predict the result?

```
while ( 1 == 1 ){
  robotLeft();
   delay(100);
    robotRight();
   delay(100);
}
```

Presented **`do`**-**`while`** loop is not an useful example as the condition `( 1 == 1 )` will never change and will be always **`true`**. So, we created an infinite loop. `Do-While` loop is far more usable if in the condition is some sensor's value, as we will see in next sections.

1.5 Questions:

1. Name the situation where `for-next` loop can be used.
2. What is the purpose of a `counter` in `for-next` loop?
3. What is the difference between `for-next` and `do-while` loops?

1.6 Summary:

1.6.1 For-loop

<++>

1.7 Issues:

1.7.1 *Can I measure the execution time of the loop?*

Yes, you can. You must save the time before the loop and save the time after the loop is executed. The difference in these two values is the spent in the execution of the loop. A minimal working example counld look like this:

```
unsigned long start_time = millis();
for (int i = 0; i<100; i++)
{
  //some code in this loop
}
unsigned long stop_time = millis();
unsigned long loop_duration = stop_time - start_time;
```